

✔ FastAPI, SQLAlchemy, Alembic - Interview Questions and Answers

This document contains curated interview questions and answers based on the technologies used in your proof-of-concept FastAPI project including routing, database handling via SQLAlchemy, migrations with Alembic, and testing setup.

◆ FastAPI

1. What is FastAPI and why is it used?

Answer: FastAPI is a modern, high-performance, Python web framework for building APIs based on standard Python type hints. It offers automatic OpenAPI documentation, high speed (comparable to NodeJS and Go), and built-in support for validation and serialization.

2. How do you define a route in FastAPI?

```
from fastapi import FastAPI

app = FastAPI()
```

```
@app.get("/ping")

def ping():

    return {"message": "pong"}
```

3. How does FastAPI handle request validation?

Answer: FastAPI uses Pydantic models to validate and parse request bodies, query parameters, path parameters, headers, etc.

```
from pydantic import BaseModel

class School(BaseModel):

    school_id: str

    school_name: str
```

4. How do you return custom HTTP status codes?

```
from fastapi import status, HTTPException

raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Item not found")
```

◆ SQLAlchemy (ORM)

5. What is SQLAlchemy?

Answer: SQLAlchemy is the most widely used SQL toolkit and Object Relational Mapper (ORM) in Python. It provides full power and flexibility of SQL.

6. How do you define a model using SQLAlchemy?

```
from sqlalchemy.orm import DeclarativeBase, mapped_column, Mapped
from sqlalchemy import String
```

```
class Base(DeclarativeBase):
```

```
    pass
```

```
class School(Base):
```

```
    __tablename__ = "ss_table_school"
```

```
    school_id: Mapped[str] = mapped_column(String, primary_key=True)
```

7. What are Mapped and mapped_column?

Answer: Mapped[...] is a generic type hint to indicate ORM-mapped fields. mapped_column() declares column properties (type, constraints).

◆ Alembic (Migrations)

8. What is Alembic?

Answer: Alembic is a lightweight database migration tool for use with SQLAlchemy. It helps track and apply schema changes.

9. How do you generate and apply a migration in Alembic?

```
alembic revision --autogenerate -m "create school table"
```

```
alembic upgrade head
```

10. What is the purpose of env.py in Alembic?

Answer: It configures the migration environment. It loads DB URL, metadata, and sets up the migration context.

◆ Routing and Dependency Injection

11. How do you use path and query parameters?

```
@app.get("/schools/{school_id}")
```

```
def get_school(school_id: str, country: str = "India"):
```

```
    ...
```

12. How does FastAPI support dependency injection?

```
from fastapi import Depends
```

```
def get_db():
    db = SessionLocal()

    try:
        yield db
    finally:
        db.close()

@app.post("/schools")
def create_school(school: School, db: Session = Depends(get_db)):
    ...
```

◆ Testing FastAPI Apps

13. How do you write unit tests for FastAPI?

Answer: Use TestClient from fastapi.testclient to simulate requests. Example:

```
from fastapi.testclient import TestClient
from app.main import app
client = TestClient(app)
```

```
def test_ping():
    response = client.get("/ping")
    assert response.status_code == 200
```

14. How do you test with a separate test database?

- Use .env.test for test DB URL
 - In conftest.py, override dependencies
 - Setup create_all() or Alembic upgrade before test
-

◆ Miscellaneous

15. How do you manage configurations?

- Use .env files with python-dotenv
- Load with os.getenv() or pydantic.BaseSettings

16. What is the role of `Base.metadata.create_all()`?

Answer: It creates tables in the database based on models if they don't already exist. Used often in tests.

17. How do you run a FastAPI app with Uvicorn in PowerShell?

```
uvicorn app.main:app --reload
```

◆ Common Issues

18. What does the error `Could not determine revision id from filename...` mean?

Answer: Your Alembic `script.py.mako` was empty or corrupted. It must contain a valid revision, `down_revision`, and `upgrade/downgrade` logic.

19. How do you fix interpolation errors in `.ini` files?

Answer: Avoid using `%(...)` syntax in `alembic.ini` or directly pass DB URL via `env.py` using `os.getenv("DATABASE_URL")`

20. How to suppress deprecation warnings in tests?

Answer: Add the following to `pytest.ini`:

```
[pytest]
filterwarnings =
    ignore::DeprecationWarning
```

Let me know if you'd like:

- Company-specific versions (GCP Cloud SQL, etc.)
- More hands-on or scenario-based Q&A
- Conversion to PDF or Markdown